

An Introduction to Cryptography

Cyber Security | 19.03.2026

Gerald Dißbaur | Lecturer

A-SIT Secure Information Technology Center Austria

www.dhbw-stuttgart.de

Agenda

Introduction

Basics about Symmetric Cryptosystems

Fundamental Principles about Asymmetric Cryptography and PKI

Cryptographic Hash Functions and Symmetric Authentication

Summary

Agenda

Introduction

Basics about Symmetric Cryptosystems

Fundamental Principles about Asymmetric Cryptography and PKI

Cryptographic Hash Functions and Symmetric Authentication

Summary

CAN CRYPTOGRAPHY REDUCE THE ATTACK SURFACE IS CRYPTOGRAPHY IN PRACTICAL APPLICATIONS USEFUL?

Cryptographic techniques are the most recognized security controls in terms of Technology to protect assets.

Cryptographic keys are also Assets (*e.g., secondary assets*) in terms of information security.

Adversaries attack data and cryptosystems

WHAT ARE THE MAIN OBJECTIVES OF ATTACKS?

3 SELECTED OBJECTIVES

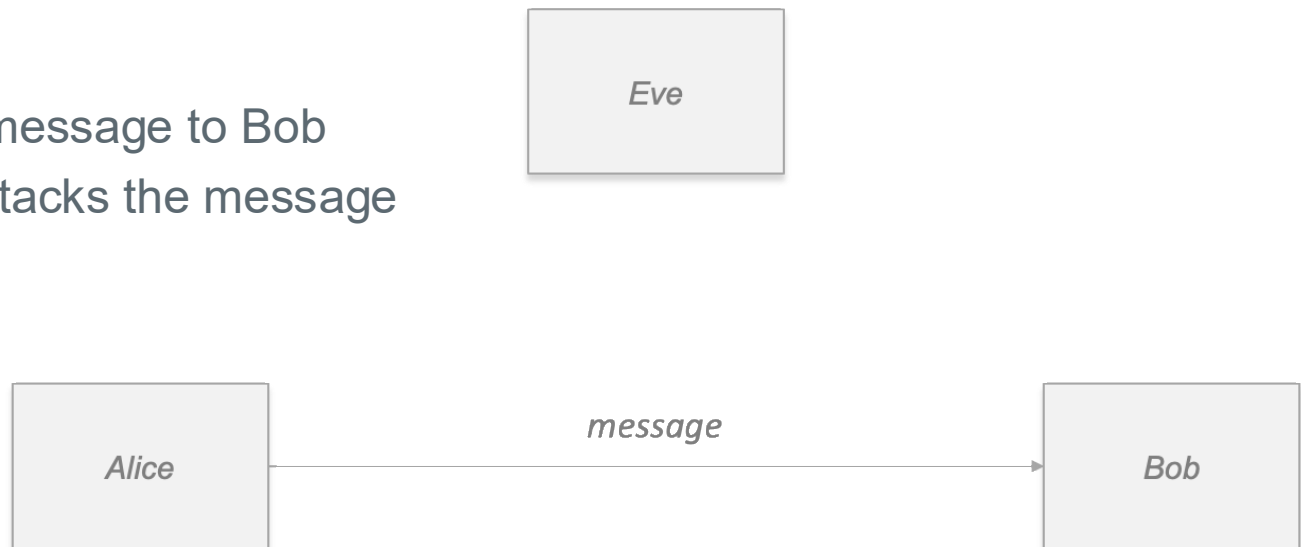
- Distinguishing attack
(i.e., broken, if an attack is found such that the encrypted data can be distinguished from random data)
- Partial compromise
(e.g., insert bits)
- Total compromise
(broken, if an attack is found such that the secret can be obtained)

Attacks are conducted by malicious adversaries such as “Eve”

THE GENERAL COMMUNICATION MODEL

ALICE | BOB | EVE THE ADVERSARY

- Alice (*the sender*)
- Bob (*the receiver*)
- Eve (*the adversary*)
- Alice transmits a message to Bob
- Eve maliciously attacks the message



WHICH ATTACK MODELS ARE RELEVANT?

INCREASINGLY POWERFUL ATTACKS

Ciphertext only attack

- The attacker only knows the ciphertext
- The attacker intercepts the ciphertext on the channel

Known plaintext attack

- The attacker knows the plaintext (*i.e., the secret message*)
- The attacker knows the ciphertext (*i.e., the encrypted form of the secret message*)

Chosen plaintext attack

- The attacker e.g., owns the cryptosystem
- The attacker can choose the plaintext
- The attacker is able to derive the ciphertext and others (*e.g., exhaustive key search, related key*)

HOW SECURE ARE CRYPTOGRAPHIC PRIMITIVES?

COMPUTATIONAL | PROVABLE | UNCONDITIONAL

Computational security (*or hardness*)

- Resistant against computationally bounded attackers
- Resistant against bounded attackers in terms of knowledge

Provable security (*or hardness*)

- Resistant against computationally bounded attackers
- There exist mathematical proofs

Unconditional security (*or hardness*) (*i.e. information theoretic*)

- Resistant against an arbitrary instance of an attacker
- The ciphertext provides no information about the plaintext

e.g.,
One-Time-Pad

Logical Operations are Performed on Data

LOGICAL OPERATIONS ON DATA

CLASSICAL BIT OPERATIONS

- Negotiate on a symmetric key
- Keylength = length of cleartext
- **One-Time Pad** (*or Vernam scheme*)
- Claude Elwood Shannon

X	Y	NOT(X)	NOT(Y)	AND	OR	NAND	NOR	XOR
0	0	1	1	0	0	1	1	0
0	1	1	0	0	1	1	0	1
1	0	0	1	0	1	1	0	1
1	1	0	0	1	1	0	0	0

Which operation might be a good choice for encryption to use One-Time Pads?

LOGICAL OPERATIONS ON DATA

CLASSICAL BIT OPERATIONS

- Negotiate on a symmetric key
- Keylength = length of cleartext
- **One-Time Pad** (*or Vernam scheme*)
- Claude Elwood Shannon
- Apply XOR operation on cleartext
- $X + Y \bmod 2$

X	Y	NOT(X)	NOT(Y)	AND	OR	NAND	NOR	XOR
0	0	1	1	0	0	1	1	0
0	1	1	0	0	1	1	0	1
1	0	0	1	0	1	1	0	1
1	1	0	0	1	1	0	0	0

**What happens if we perform XOR multiple times
and if we apply the keys in a different order?**

THE ONE-TIME PAD

PROVABLE SECURE

- Alice (*the sender*) and Bob (*the receiver*) wish to communicate in a private manner
- Alice and Bob share a private key $K \in \mathcal{A}\{0,1\}^n$
- Alice sends a cleartext message $M \in \mathcal{A}\{0,1\}^n$ to Bob in an encrypted manner
- Alice computes $E = K \oplus M$ (*logical XOR-operation*)
- Bob receives E and derives $M = K \oplus E = K \oplus K \oplus M$

The One-Time Pad is kept secret and it is only used once to protect a message with a random key of the same length

EXAMPLES FOR ONE-TIME PAD | THE SENDER ENCRYPTS

BITWISE XOR OPERATION | ENCRYPTION

1011	1110	1010	0011	0101	0100	1101	0011	Plaintext P
1001	0000	1100	1010	0001	0110	1110	1010	Key K_1
0010	1110	0110	1001	0100	0010	0011	1001	Ciphertext C_1

1011	1110	1010	0011	0101	0100	1101	0011	Plaintext P
1001	0000	1100	1010	0001	0110	1110	1010	Key K_1
1011	0110	1101	0100	0100	1111	1001	0110	Key K_2
1001	1000	1011	1101	0000	1101	1010	1111	Ciphertext C_2

EXAMPLES FOR ONE-TIME PAD | THE SENDER DECRYPTS BITWISE XOR OPERATION | DECRYPTION

0010	1110	0110	1001	0100	0010	0011	1001		Ciphertext C_1
1001	0000	1100	1010	0001	0110	1110	1010		Key K_1
1011	1110	1010	0011	0101	0100	1101	0011		Plaintext P

1001	1000	1011	1101	0000	1101	1010	1111		Ciphertext C_2
1011	0110	1101	0100	0100	1111	1001	0110		Key K_2
1001	0000	1100	1010	0001	0110	1110	1010		Key K_1
1011	1110	1010	0011	0101	0100	1101	0011		Plaintext P

It does not matter in which order you apply the Keys K_i for XOR operations

DATA TRANSMISSION AND STORAGE

3 ROLES | ALICE, BOB, EVE

The main objective in cryptography is to securing the communications between Alice and Bob.

- Alice is a the sender
- Bob is a receiver
- Eve is a malicious attacker (*or: eavesdropper*).

We can distinguish 2 communication models

- 1) Data Transmission and/or Processing (*Alice to Bob*)
- 2) Data Storage (*Alice to Alice but delayed in terms of time*)

WHAT DOES ENCRYPTION AND DECRYPTION MEAN?

ALPHABET, PLAINTEXT, CIPHERTEXT, KEY SPACE

Elements of Encryption

A ... Alphabet (*e.g., binary $\{0,1\}$ finite set of symbols*)

M ... Message space (*contains symbols from an alphabet such as A_1 ;
elements are also known as P for plaintext*)

C ... Ciphertext (*contains symbols from an alphabet such as A_2 ;
elements are also known as ciphertext messages*)

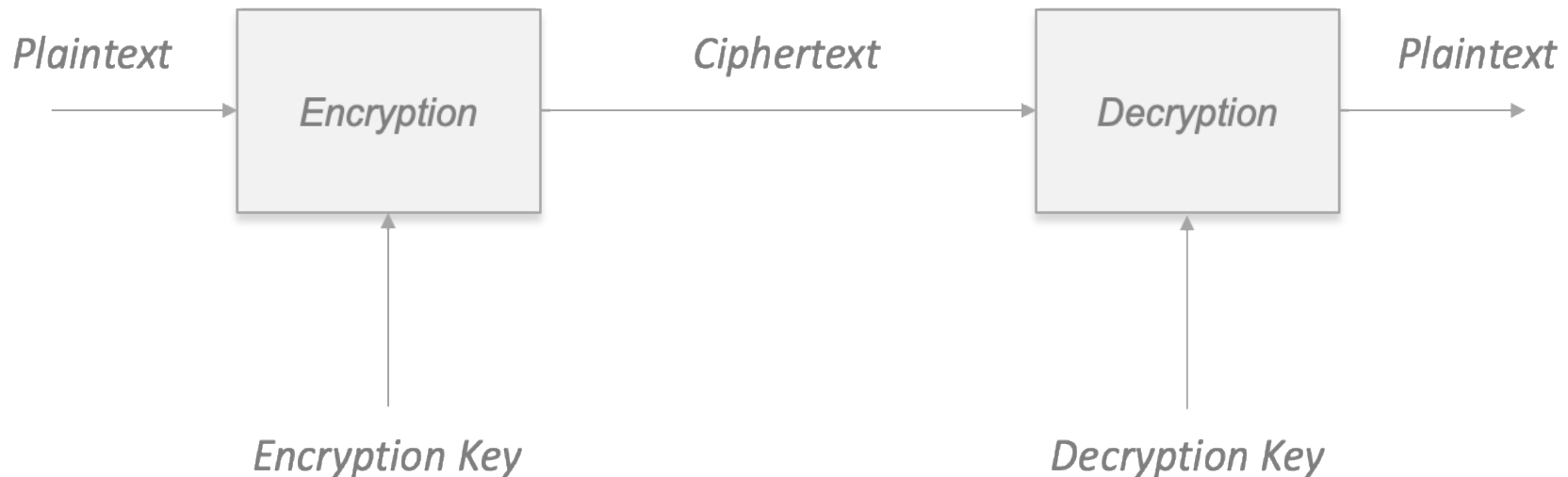
K ... Keyspace

- Encryption function i.e., a Bijection $E: M \rightarrow C$
- Decryption function i.e., a Bijection $D: C \rightarrow M$

KERCKHOFFS' PRINCIPLE

KEEPING KEY MATERIAL SECRET

A cryptosystem should be secure even if everything (e.g., *cryptographic primitive*¹, *cryptographic protocol*²) except the secret key is publicly available.



¹ Cryptographic primitive: An algorithm to perform cryptographic operations

² Cryptographic protocol: How to apply the cryptographic primitive

PROPERTIES OF CRYPTOGRAPHIC PRIMITIVES

PUBLIC | BLACK BOX | COMPUTATIONALLY BOUNDED

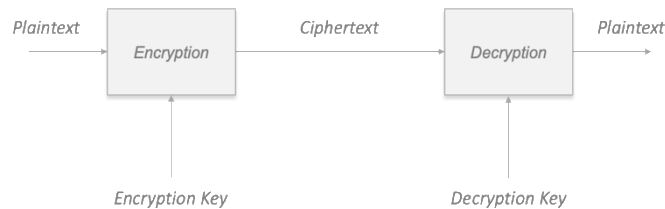
- Primitives (*i.e., algorithms*) are publicly available (*cf., Kerckhoffs' principle*)
- Deal cryptographic primitives as black boxes
- Compose systems based upon multiple primitives
- The security is mainly based upon unproven assumptions concerning the hardness of certain mathematical problems (*e.g., discrete logarithm, prime factorization*) which means it is computationally infeasible to break

CRYPTOGRAPHY USES ALSO KEY MATERIAL

ENCRYPTION AND DECRYPTION OPERATION

2 Options for keys

- Different keys
- Same keys



$$E_{K,1}(P) = C$$

$$D_{K,2}(C) = P$$

$$D_{K,2}(E_{K,1}(P)) = P$$



$$E_K(P) = C$$

$$D_K(C) = P$$

$$D_K(E_K(P)) = P$$

WHICH TYPES OF CRYPTOSYSTEMS EXIST?

2 FAMILIES SYMMETRIC VS. ASYMMETRIC CRYPTOSYSTEMS

Symmetric key cryptography

- How can Alice store data on e.g., a HDD/SSD such that the confidentiality is assured?
- Assume that a shared secret key between Alice and Bob exists. How can Alice transmit an encrypted message to Bob such that only Bob can decipher it?

Asymmetric key cryptography

- How can Alice encrypt a document such that only Bob can decrypt it?
- How can Alice sign a document such that Bob can deduce that the document has been signed by Alice?
- How can Alice and Bob negotiate a secret key?

HOW CAN WE DIFFERENTIATE THOSE 2 FAMILIES?

SYMMETRIC VS. ASYMMETRIC CRYPTOSYSTEMS

Symmetric key cryptography

- All key material is available and known to both Alice and Bob
- Sender and receiver can be swapped
- Same key for encryption, and for decryption or the encryption key can be derived from the encryption key and vice versa

Asymmetric key cryptography

- Key pairs are applied (*i.e.*, *public*, *secret*) are associated to each other
- Different key for Alice and Bob; Secret keys are exclusively known to owner
- Different keys for Alice and Bob are used
- Difficult primitives, simpler protocols
- Decryption keys cannot be derived from the encryption key (*in reasonable time*)

2 GROUPS OF SYMMETRIC CRYPTOSYSTEMS

BLOCK CIPHERS AND STREAM CIPHERS

2 different kinds of symmetric cryptosystems

(1) **Stream ciphers.** Operating on the plaintext a single bit / byte

- Stateful or counter based
- Hardware-oriented stream ciphers (historically cheaper than block ciphers) e.g., A5/1 (broken)
- Software-oriented stream ciphers e.g., RC4 (broken)

(2) **Block ciphers.** Operating on the plaintext in groups of bits / byte

Cryptosystems are used to perform both encryption and decryption.

WHICH SECURITY OBJECTIVES ARE RELEVANT?

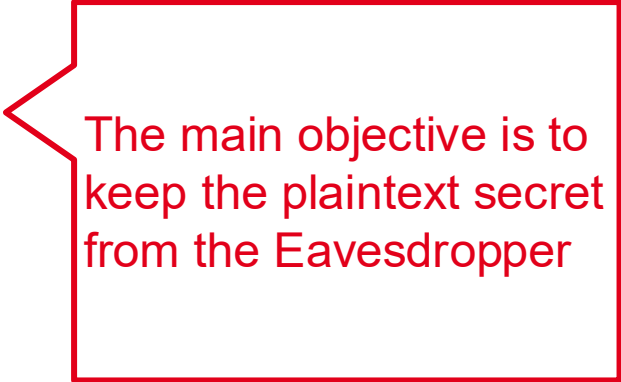
CONSIDER CIA-TRIADE FROM INFORMATION SECURITY

Recap

- Confidentiality
- Integrity
- Availability

and also

- Anonymity
- Contract signing
- Non-repudiation
- Key escrow
- Time stamping



The main objective is to keep the plaintext secret from the Eavesdropper

THE VERNAM SCHEME

ALSO KNOWN AS ONE-TIME-PAD

General observations

- The Vernam scheme is also known as „*One-Time-Pad*“
- The keys (*i.e., each character*) are only used once
- The keys are computed on a random basis
- The length of the key equals the length of the plaintext
- The ciphertext does not give information about the message / key

Long Keys
are not very
practical.

P, C, K are arbitrary instances of strings over the alphabet $\{0, 1, \dots, l-1\}$

Encryption:

$$C_i = P_i + K_i \bmod L$$

$$i = 1, \dots, n$$

***The Vernam Scheme provides perfect Security
(but not in terms of key negotiation)***

OVERVIEW ABOUT CRYPTOGRAPHIC PRIMITIVES

SYMMETRIC, ASYMMETRIC, HASH-FUNCTIONS

Symmetric primitives

- Symmetric-key ciphers (*Block-ciphers, Stream-ciphers*)
- Digital signatures
- Pseudo-random numbers

Asymmetric primitives (*also Public-Key-Primitives*)

- Public-key ciphers

Hash functions

- Unkeyed hash functions (*e.g., HMAC¹*)
- Keyed hash functions (*e.g., MD5², SHA³-1, SHA-2, SHA-3*)

¹ HMAC – keyed-hash message authentication code

² MD5 – message digest algorithm

³ SHA – Secure hash algorithm

RANDOM NUMBERS

BASIS IN CRYPTOGRAPHY

Everything starts with a random value

- Key generation
- Random
- Nonce
- Challenges
- and more

KINDS OF RANDOM NUMBER GENERATORS

AN OVERVIEW

Random Number Generators

- Source of entropy

Pseudo-Random Number Generators

- Cryptographic algorithm to produce high-quality random bits from source of entropy
- Deterministic Random Number Generators
- Deterministic Random Bit Generators

RANDOM NUMBERS | cont´d

AN OVERVIEW

Input from the environment

- Physics
- Information from operating systems

Software and hardware components

True Random Number Generators (TRNG)

Quantum Random Number Generators (QRNG)

RANDOM NUMBERS | cont'd

AN OVERVIEW

Deterministic Random Bit Generators (DRBG)

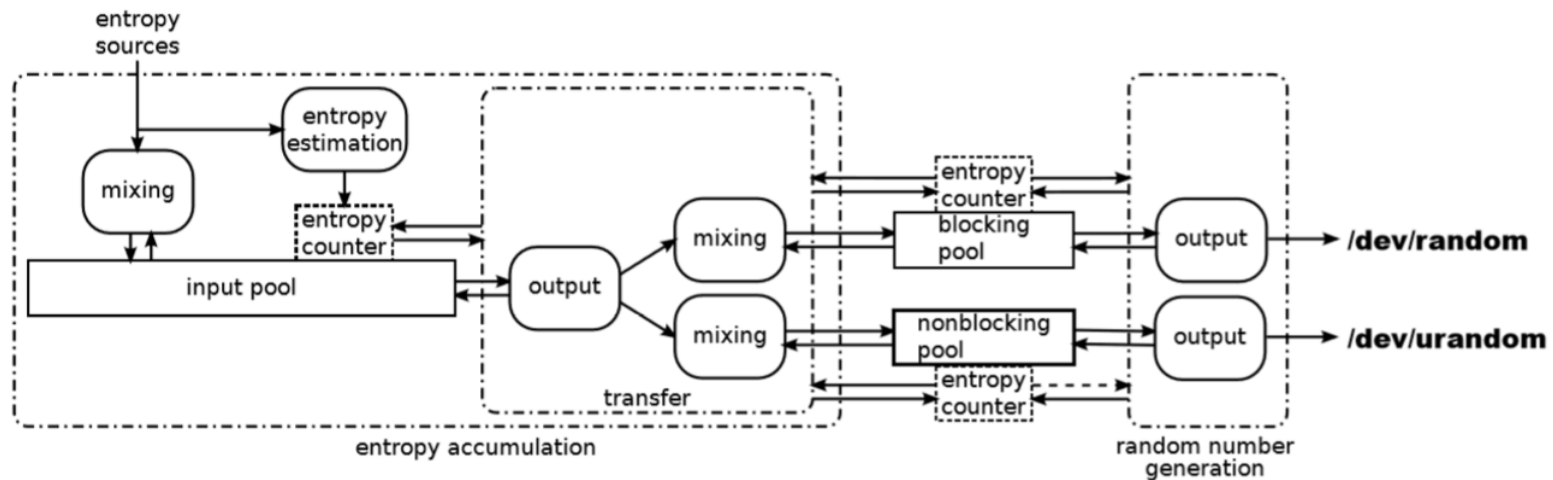
- Implementation of PRNG
- Utilizes cryptographic algorithms
 - Usually implementsInit()
 - Refresh()
 - Next()

How can we build DRBG using:

- Stream ciphers
- Block ciphers in CTR mode
- Hash functions
- MACs

RANDOM NUMBERS

SELECTED EXAMPLE | ANDROID



Agenda

Introduction

Basics about Symmetric Cryptosystems

Fundamental Principles about Asymmetric Cryptography and PKI

Cryptographic Hash Functions and Symmetric Authentication

Summary

GENERAL OBSERVATIONS ABOUT BLOCK CIPHERS

ENCRYPTION FUNCTIONS FOR FIXED BLOCK SIZES

Block ciphers are a central building block in cryptography

Practical applications of block ciphers are

- Authentication mechanisms / schemes
- Authenticated encryption schemes
- Symmetric encryption systems
- One-way functions
- Pseudo-random number generators (*e.g., used in HSMs*)

In general, block ciphers are present in every day practical applications

LET US START WITH STREAM CIPHERS

BLOCK CIPHERS | BLOCK LENGTH $T = 1$

Properties of stream ciphers

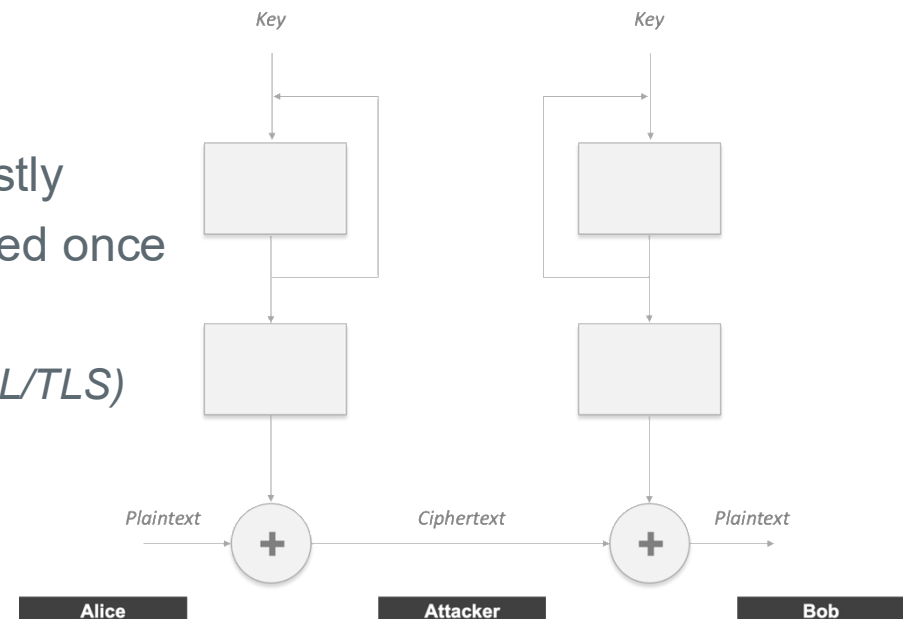
- Each bit is differently substituted
- Vernam cipher
- Key streams are pseudo random, mostly
- One-time-pad if e_i are random and used once

Practical examples

- RC4 (used in legacy WEP¹, WPA², old SSL/TLS)
- Attention! RC2, RC5, RC6 are no stream ciphers
- Salsa / ChaCha20 (e.g., TLS 1.3)

¹ WEP – Wired equivalent privacy

² WPA – WiFi Protected Access



BLOCK CIPHERS

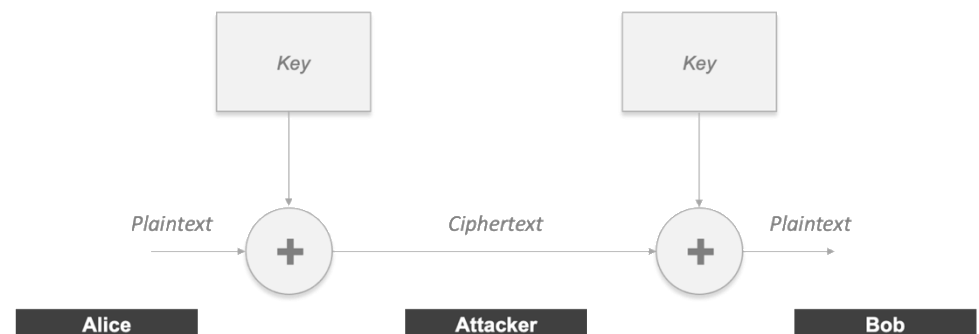
GENERAL ARCHITECTURE | VERNAM

Vernam scheme

Encryption is addition modulo 26

Examples

- $G + D \bmod 26 = K$
- $Q + T \bmod 26 = K$
- $L + O \bmod 26 = K$
- $K + V \bmod 26 = G$



The sender uses exactly one character of the key to encrypt exactly one character of the plaintext. The receiver uses each character to decrypt.

BLOCK CIPHERS

SELECTED EXAMPLES

Extensively used in practical applications (*e.g., Smartphones, databases*)

- **AES**¹
- Camellia²
- DES³
- TDES⁴

Key negotiation (or key distribution) remains a problem.

¹ AES – Advanced Encryption Standard

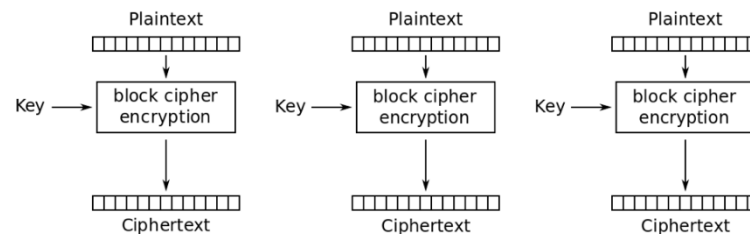
² Camellia – Developed by Mitsubishi and NTT

³ DES – Data Encryption Standard

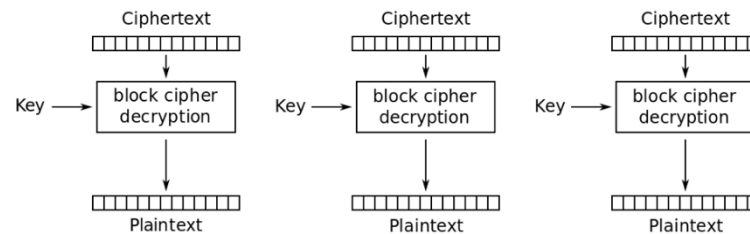
⁴ TDES – Triple DES

MODES OF OPERATION

ECB – ELECTRONIC CODE BOOK MODE



Electronic Codebook (ECB) mode encryption

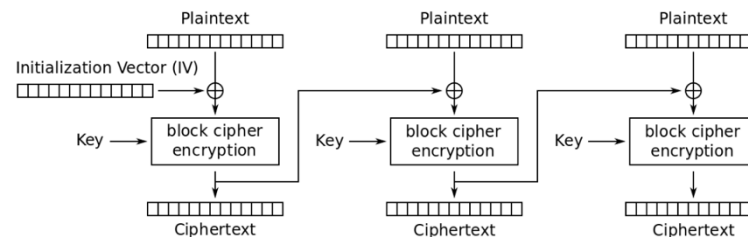


Electronic Codebook (ECB) mode decryption

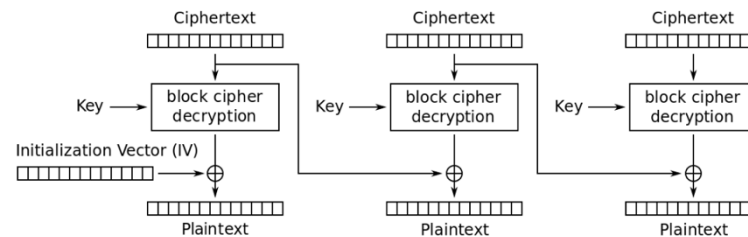
¹ Source: Wikipedia

MODES OF OPERATION | cont'd

CBC – CIPHER BLOCK CHAINING MODE



Cipher Block Chaining (CBC) mode encryption

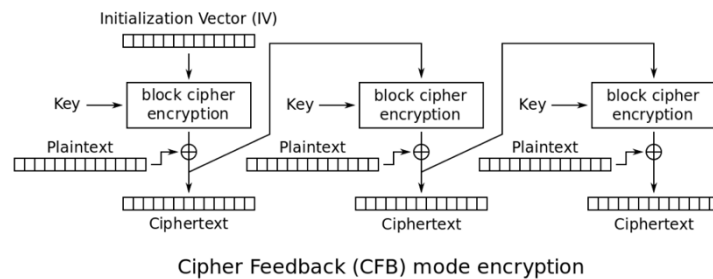


Cipher Block Chaining (CBC) mode decryption

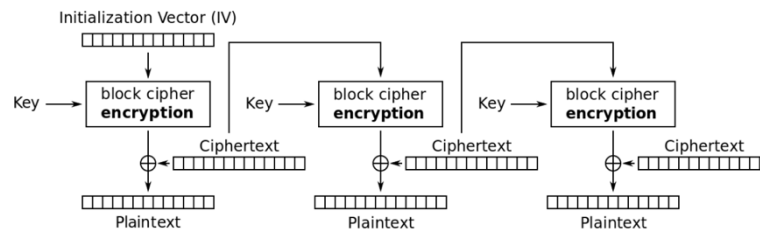
¹ Source: Wikipedia

MODES OF OPERATION | cont'd

CFB – CIPHER FEEDBACK MODE



Cipher Feedback (CFB) mode encryption

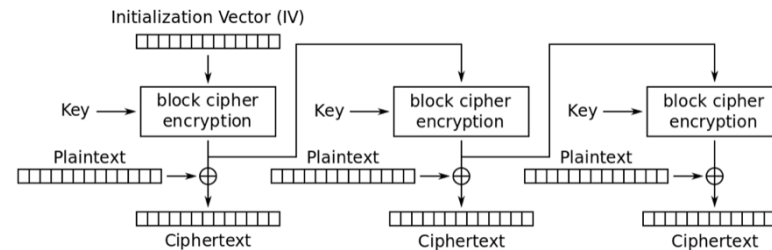


Cipher Feedback (CFB) mode decryption

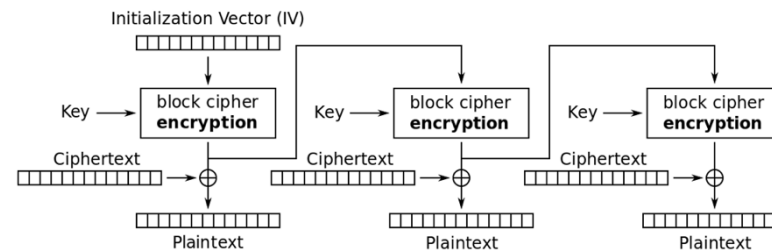
¹ Source: Wikipedia

MODES OF OPERATION | cont'd

OFB – OUTPUT FEEDBACK MODE



Output Feedback (OFB) mode encryption

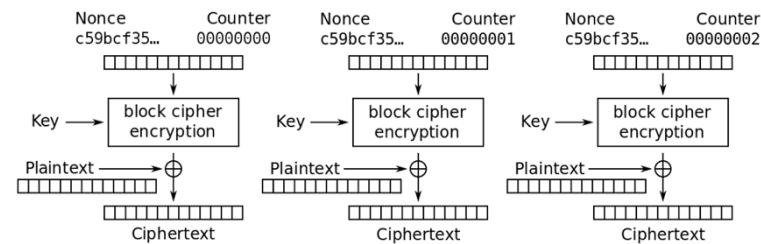


Output Feedback (OFB) mode decryption

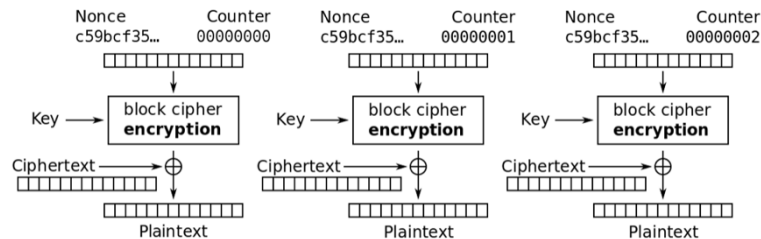
¹ Source: Wikipedia

MODES OF OPERATION | cont'd

CTR – COUNTER MODE



Counter (CTR) mode encryption



Counter (CTR) mode decryption

¹ Source: Wikipedia

BLOCK CIPHERS VS. STREAM CIPHERS

SYMMETRIC CRYPTOSYSTEMS

Block ciphers

- Segment plaintext into fixed length blocks (*i.e., strings of size t*)
- Substitution and permutation (transposition)
- Encrypt one block per time-unit

Stream ciphers

- Also a block cipher, but with length of block $t = 1$
- Different key streams are used

WHICH ENCRYPTION MODES ARE RELEVANT?

ENCRYPTION MODES FOR BLOCK CIPHERS

Block cipher encrypts blocks of fixed size

- AES. 128, 192 or 256 bit keylength, 128 bit block size
- DES. 56 bit key, 64 bit block size
- [...]

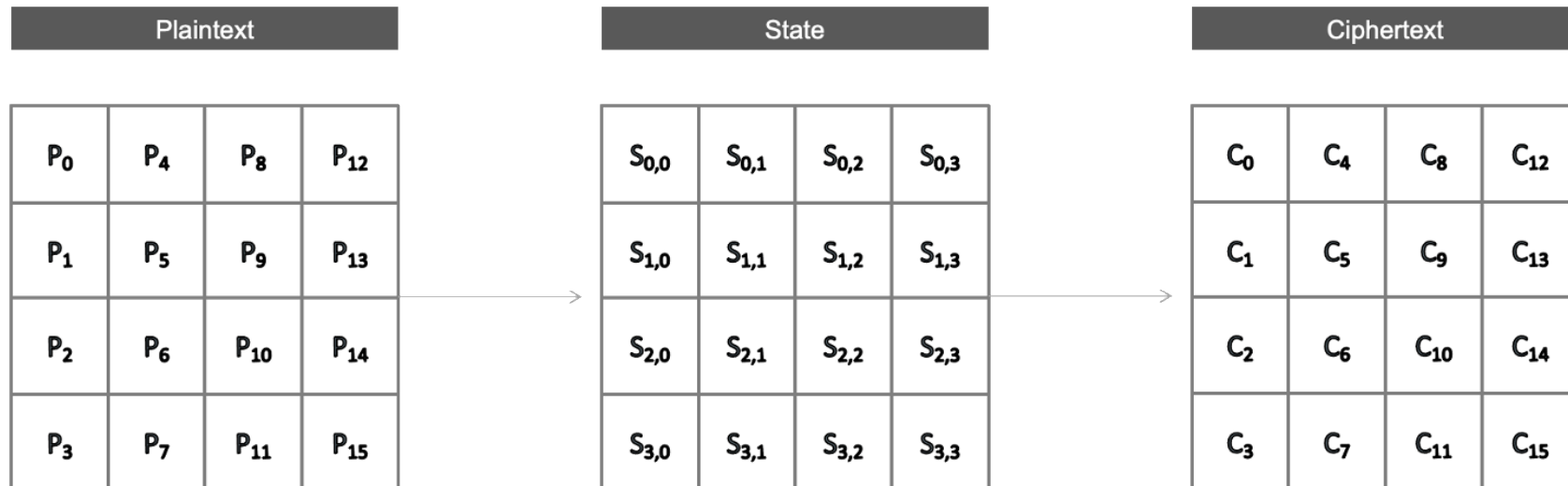
But what to do with block lengths that are not of fixed length?

- Cipher Block Chaining (*CBC*)
- Cipher Feedback (*CFB*)
- Electronic Code Book (*ECB*)
- [...]

WHAT IS AN AES-STATE?

4x4 MATRIX

- 4x4 matrix (*plaintext, intermediate state, ciphertext*)
- Matrix representation provides transformations for AES



WHICH 4 AES-TRANSFORMATIONS ARE POSSIBLE?

ADDROUNDKEY | SUBBYTES | SHIFTRROWS | MIXCOLUMNS

AddRoundKey (*bits*)

- Bitwise XOR between a round key and the state

SubBytes (*bytes*)

- Maps an input byte of the State to a new output byte of the state
- The mapping is based on an inversion and called Sbox
- In software usually implemented as table lookup

ShiftRows (*rows*)

- Rotates the rows of the state to the left
- Offsets for the different rows: 0, 1, 2, 3

Mixcolumns (*columns*)

- Polynomial multiplication of each column with a constant polynomial
- Most complex operation of AES software implementations

Agenda

Introduction

Basics about Symmetric Cryptosystems

Fundamental Principles about Asymmetric Cryptography and PKI

Cryptographic Hash Functions and Symmetric Authentication

Summary

KEY NEGOTIATION IN PUBLIC KEY SYSTEMS

6 STEPS

- 1) Alice and Bob negotiate to use an asymmetric cryptosystem
- 2) Bob transmits his public key to Alice (*e.g., via an unprotected channel*)
- 3) Alice encrypts her plaintext (P) by applying Bobs public key
- 4) Alice sends the ciphertext (C) to Bob (*e.g., via an unprotected channel*)
- 5) Bob receives the ciphertext
- 6) Bob decrypts Alice's ciphertext by applying his kept secret private key

No prior negotiations required. Alice generates a random key.

SIGNATURE SCHEMES

CRYPTOGRAPHICALLY ASSOCIATED KEY PAIRS

Basic architecture

- Key pair (K_V, K_S)
- K_S is the secret signature key that is kept private
- K_V is the signature verification key that is publicly available

Signature creation

$$S = \text{Sign}_{K_S}(H(M))$$

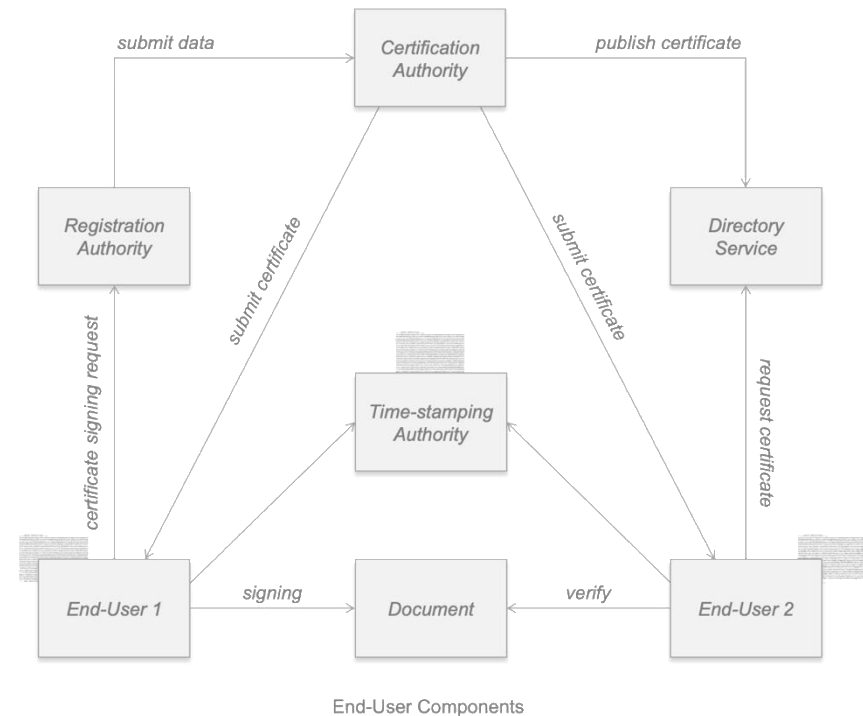
Components

- Sign_{K_S} – Signature function applying K_S
- S – Signature (*i.e., digital signature*) with fixed size
- M – Message (*i.e., not necessarily plaintext*) with arbitrary size
- $H(M)$ – Hash function over the message

WHAT ARE THE BUILDING BLOCKS OF A PKI

THE BASIC ARCHITECTURE OF A PKI

- End-user components
- Certificates
- Certification Authority (CA)
- Organizational policies
- Registration Authority (RA)
- Revocation data (*CRL*¹/*OCSP*²)
- Time-stamping Authority (TSA)
- Directory Service (*DIR*)



¹ CRL – Certificate Revocation List

² OCSP – Online Certificate Status Protocol

³ PEM – Privacy Enhanced Mail

PKI & CERTIFICATION AUTHORITY

INTERNET TRUST MODEL

Public Key Infrastructure

An infrastructure that allows one to recognize which public key belongs to whom

Certification Authorities

are one of the most important parts of the current internet trust model

- Issue a certificate for any domain name
- Has have a lot of power and a lot of responsibilities

THE MAIN COMPONENTS OF A PKI

SAMPLE CERTIFICATE

- End-user components
- Certificates (*PEM very often used*)
- Certification Authority (CA)
- Organizational policies
- Registration Authority (RA)
- Revocation data (*CRL¹/OCSP²*)
- Time-stamping Authority (TSA)
- Directory Service (*DIR*)

PEM³-File

-----BEGIN CERTIFICATE-----

```
0815UdMhd6OgAwIBAgIDFE31MA0GCSqGSIb3DQEBBQUAMIGLMQswCQYDVQQGEwJB
VDFIMEYGA1UECgw/QS1UcnV83736hMKI8dGg2kdpY2hlcmhlaXRzc3lzdGVtZSBp
ki8dejweiok2098dDK2W9XwKn5+rUJDbQc5WgHkqWP2TU3qcW/NZYSSIIY6PVG06b
hdsdnOyMPdrBhnpvNi4m63/zwjzhjnfjdcoKyQg/youwpVkePZ+e8032hf9nqvt
U4VDig+gj+6kjjkdAEC+wjmk982jK98+HyLTDLRQ608p5HI3Sov4FuwiMgcqIyzz
VHJ1c3QtUXVhbC0wMzCCASIwDQYJokwujsj92773839dkdnPADCCAQoCggEBALoe
okuHXXMuIGYuIFNpY2hlklajshdfb3lzdGVtZSBpbSB1bGVrdHIuIERhdGVudmVy
jA/CmId9oz8sy4rWEklti+8TLS/h98762mdkwjdiuzDZmz4Qh4fLFAAEv8iGSZiW
o9fNyqmnT3K/vBGOqBsCAwEAAAM2MDQwDwYDVDR0TAQH/BAUwAwEB/zARBgNVHQ4E
CgQIRgbfN/LCNxAwDgYDVDR0PAQH/BAQDAgEGMA0GCSqGSIb3DQEBBQUAA4IBAQB
bSB1bGVrdHIuIERhdGVudmVya2VociBHbWJIMRgwFgYDVQQLDA9BLVRydXN0LVF1
j28kdDJ23pskdDw1U0UwmMkjhgbe2dYDVQQGEwJBVDFIMEYGA1UECgw/QS1UcnVz
L4wZohydjMwzcHUJB/5e+0CkJYwv/bVRZzqoK0Yf1midYJukxW0w7nDNKD9KdOpZ
rGb2GrVdLIesuvivSP9zq09M8bcm3jDPCHv5bcudPZVX7rkr79VphgLfKl1NA4Fw
+XXFAAZuH7BjwSsqEl38y9Jlki(3hd/zeHJdGfFLwr7OHIGbDPEmQXcyOcaMUGjP
a2VociBHhzede/gwFgYdez6dDA9BLVRydXN0LVF1YWwtMDMxGDABGnVBAMMD0Et
+7+Gv7HTcuEdeJDUE/o2kJLSQA/3KBj1ttHXLiG943wnvcZf3LtkFc+8Ia5N3bNS
BM6q7/OFKHSS+iHCAJ8XS6SnLITYRyfo0thsXBahpkTUEP0uJY+yoE/EcBkvsGX
drLIawu8YOrNZvs3SjhawefdklsiqiNtLUGWRxpLwssfBho5K0NqpX987329kHbD
Vxki983mKJH3fANQ0815
```

-----END CERTIFICATE-----

¹ CRL – Certificate Revocation List

² OCSP – Online Certificate Status Protocol

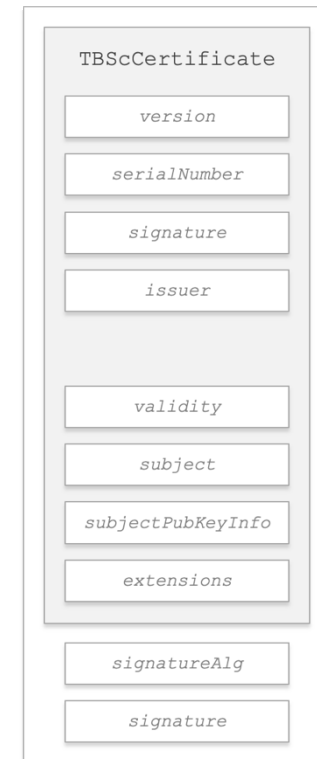
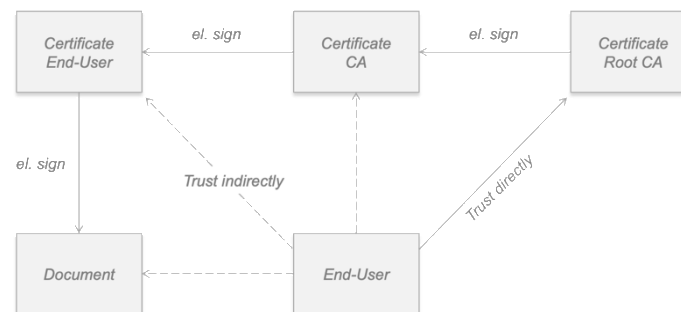
WHAT IS THE BASIC STRUCTURE OF A X.509 CERTIFICATE?

GENERAL STRUCTURE AND EXAMPLE

- Digital certificates associate a public key to components
- Standard x.509
- Commonly used version 3 i.e., x.509v3
- PEM is a superset of x.509

Certificate chain

- RootCA
- Intermediate CA
- End-User
- Certificates
- Document



X.509 CERTIFICATE BASIC STRUCTURE

CertificateVersion Number

- Serial Number
- Signature Algorithm ID
- Issuer Name
- Validity period
- Not Before
- Not After
- Subject name
- Subject Public Key Info
- Public Key Algorithm
- Subject Public Key
- Issuer Unique Identifier (optional)
- Subject Unique Identifier (optional)
- Extensions (optional)

Certificate Signature Algorithm

Certificate Signature

X.509 CERTIFICATE

BINDING, ISSUING, CONTROL

Certificate binds

- Entity identifier
- Public Key

Certificates are issued

- Issuer verifies binding
- Issuer signs certificate
- User has to trust issuer

Key owner has

- Exclusive key control

X.509 CERTIFICATE

BINDING, ISSUING, CONTROL

X.509 usually refers to the IETF's PKIX certificate and CRL Profile of the X.509 v3 certificate standard, as specified in RFC 5280

X.509 defines

- Certificate format
 - Public key certificates
 - Attribute certificates
- Revocation list format
- Certificate paths
 - Construction
 - Validation

X.509 CERTIFICATE EXTENSIONS

X.509 Extensions

- Additional properties
- Standard defined
- Proprietary extensions possible
- Key usage restrictions
- Certificate policies
- Name constraints
- CRL distribution points
- Authority information access
- Basic constraints

...

X.509 CERTIFICATES

ENCODINGS AND FORMATS

Abstract Syntax Notation One (ASN.1) is a set of rules that support definition, transport, and exchange of complex data structures and objects

Distinguished Encoding Rules (DER), which are a subset of Basic Encoding Rules (BER) that allow only one way to encode ASN.1 values.

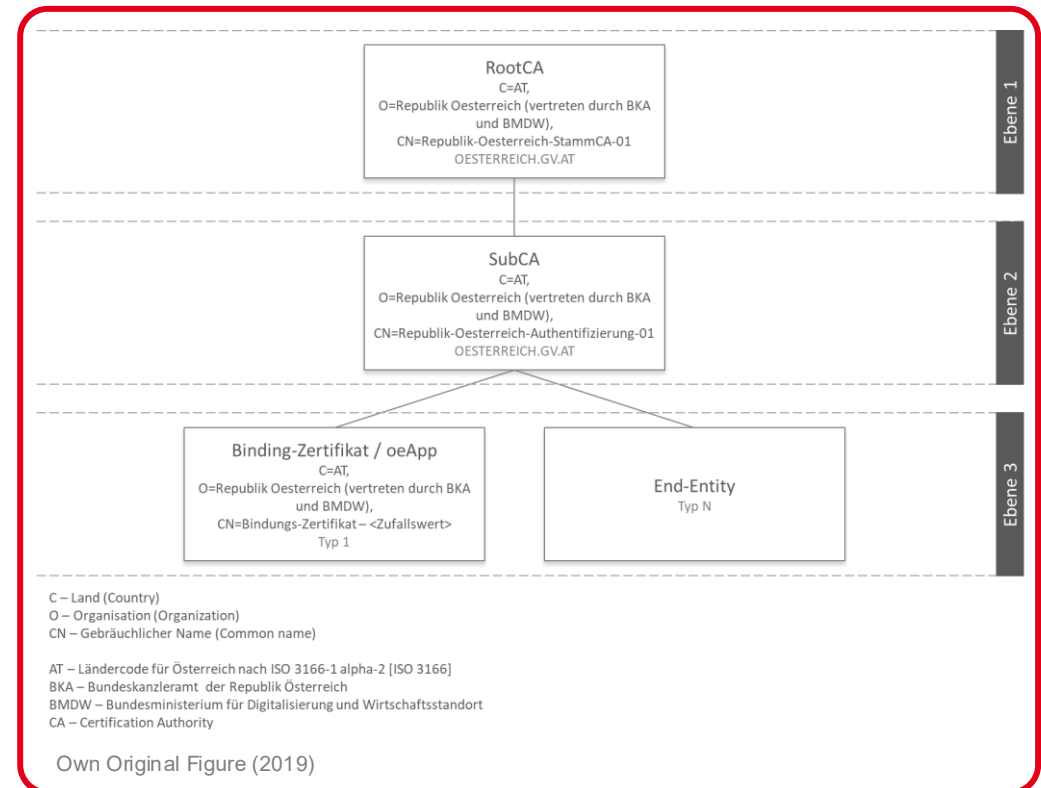
CERTIFICATE HIERARCHY | PKI OESTERREICH.GV.AT

SELECTED PRACTICAL EXAMPLE

Certification Authority (CA)

3-layered certificate hierarchy

- RootCA
 - SubCA
 - End-Entities
-
- **X.509 Certificates**
 - **SHA¹-384/ECDSA²**



¹ SHA – Secure Hash Algorithm

² ECDSA – Elliptic Curve Digital Signature Algorithm

CERTIFICATE PROFILE

SELECTED EXAMPLE | OESTERREICH.GV.AT

	OEGVAT PKI RootCA (SHA-384/ECDSA) (OEGV SCA)	OEGVAT PKI RootCA OCSP Signer (OEGV SCA OCSP)	OEGVAT PKI SubCA (SHA-384/ECDSA) (OEGV ACA)	OEGVAT PKI SubCA OCSP Signer (OEGV ACA OCSP)	OEGVAT PKI EndEntity (OEGV OEAPP CLIENT)
Verwendung	CA-Zertifikat	Signatur von OCSP Responses	CA-Zertifikat	Signatur von OCSP Responses	End-Entity-Zertifikat
Version	v3	v3	v3	v3	v3
SerialNumber	60 bit, Zufallszahl	60 bit, Zufallszahl	60 bit, Zufallszahl	60 bit, Zufallszahl	60 bit, Zufallszahl
Signaturalgorithmus	ecdsa-with-SHA384 (1.2.840.10045.4.3.3)	ecdsa-with-SHA384 (1.2.840.10045.4.3.3)	ecdsa-with-SHA384 (1.2.840.10045.4.3.3)	ecdsa-with-SHA384 (1.2.840.10045.4.3.3)	ecdsa-with-SHA384 (1.2.840.10045.4.3.3)
Aussteller	siehe Antragsteller	OEGV SCA	OEGV SCA	OEGV ACA	OEGV ACA
Gültigkeitsdauer	12 Jahre (4 382 Tage) (ASN.1 UTCTime)	15 Monate (458 Tage) (ASN.1 UTCTime)	12 Jahre (4 382 Tage) 31.12.2031 (ANS.1 UTCTime)	7 Tage (ASN.1 UTCTime)	oeAPP: maximal 6 Monate (183 Tage) (ASN.1 UTCTime)
Verwendungsdauer	7 Jahre für die Ausstellung von SubCA-Zertifikaten 12 Jahre für die Ausstellung von OCSP-Signer-Zertifikaten	1 Jahr	7 Jahre für die Ausstellung von Binding- Zertifikaten 12 Jahre für die Ausstellung von OCSP-Signer- Zertifikaten	1 Tag	Maximal 6 Monate für Handy-Signatur Authentifizierung Maximal 3 Monate für eIDAS (EU) Authentifizierung

Own Original Figure (2019)

CERTIFICATE PROFILE | cont'd

SELECTED EXAMPLE | OESTERREICH.GV.AT

	OEGVAT PKI RootCA (SHA-384/ECDSA) (OEGV SCA)	OEGVAT PKI RootCA OCSP Signer (OEGV SCA OCSP)	OEGVAT PKI SubCA (SHA-384/ECDSA) (OEGV ACA)	OEGVAT PKI SubCA OCSP Signer (OEGV ACA OCSP)	OEGVAT PKI EndEntity (OEGV OEAPP CLIENT)
Antragsteller (s. Abschnitt 3.1)	CN=Republik-Oesterreich-StammCA-01 O=Republik Oesterreich (vertreten durch BKA und BMDW) C=AT alle RDN: ASN.1 PrintableString	CN=Republik-Oesterreich-StammCA-01-OCSP-Responder O=Republik Oesterreich (vertreten durch BKA und BMDW) C=AT alle RDN: ASN.1 PrintableString	CN=Republik-Oesterreich-Authentifizierung-01 O=Republik Oesterreich (vertreten durch BKA und BMDW) C=AT alle RDN: ASN.1 PrintableString	CN=Republik-Oesterreich-Authentifizierung-OCSP-01-Responder-01 O=Republik Oesterreich (vertreten durch BKA und BMDW) C=AT alle RDN: ASN.1 PrintableString	Siehe Abschnitt 3.1.1 alle RDN: ASN.1 PrintableString wenn ausreichend; UTF8String sonst
Öffentlicher Schlüssel	ECDSA 384 Bit NIST-Kurve P-384 OID 1.2.840.10045.2.1 (id-ecPublicKey) AlgorithmIdentifier-Feld: secp384r1 1.3.132.0.34	ECDSA 384 Bit NIST-Kurve P-384 OID 1.2.840.10045.2.1 (id-ecPublicKey) AlgorithmIdentifier-Feld: secp384r1 1.3.132.0.34	ECDSA 384 Bit NIST-Kurve P-384 OID 1.2.840.10045.2.1 (id-ecPublicKey) AlgorithmIdentifier-Feld: secp384r1 1.3.132.0.34	ECDSA 384 Bit NIST-Kurve P-384 OID 1.2.840.10045.2.1 (id-ecPublicKey) AlgorithmIdentifier-Feld: secp384r1 1.3.132.0.34	ECDSA 256 Bit, NIST-Kurve P-256 OID 1.2.840.10045.2.1 (id-ecPublicKey) AlgorithmIdentifier-Feld: secp256r1 1.2.840.10045.3.1.7
Zertifikatserweiterungen					
Zertifizierungsrichtlinie (CertificatePolicies)	non-critical 1.2.40.0.10.1.13.1	non-critical 1.2.40.0.10.1.13.1	non-critical 1.2.40.0.10.1.13.1	non-critical 1.2.40.0.10.1.13.1	non-critical 1.2.40.0.10.1.13.1
Basiseinschränkungen (BasicConstraints)	critical cA=true; pathLength nicht limitiert	nicht vorhanden	critical cA=true; pathLength nicht limitiert	nicht vorhanden	nicht vorhanden
Schlüsselkennung des Antragstellers (SubjectKey Identifier)	non-critical	non-critical	non-critical	non-critical	non-critical
Schlüsselkennung des Ausstellers (AuthorityKey Identifier)	non-critical mit Attribut keyIdentifier	non-critical mit Attribut keyIdentifier	non-critical mit Attribut keyIdentifier	non-critical mit Attribut keyIdentifier	non-critical mit Attribut keyIdentifier
Key Usage	critical keyCertSign	critical digitalSignature	critical keyCertSign	critical digitalSignature	critical OEAPP CLIENT, digitalSignature
Extended Key Usage	nicht vorhanden	critical id-kp-OCSPSigning	nicht vorhanden	critical id-kp-OCSPSigning	critical OEAPP CLIENT, id-kp-clientAuth
CRL Distribution Points	nicht vorhanden	nicht vorhanden	nicht vorhanden	nicht vorhanden	nicht vorhanden
Authority Information Access	nicht vorhanden	nicht vorhanden	non-critical accessMethod id-ad-ocsp (http)	nicht vorhanden	non-critical accessMethod id-ad-ocsp (http)
NoCheck	nicht vorhanden	non-critical	nicht vorhanden	non-critical	nicht vorhanden
Alternativer Antragstellername (SubjectAlternative Name)	nicht vorhanden	nicht vorhanden	nicht vorhanden	nicht vorhanden	non-critical OEAPP CLIENT (optional) dNSName, IPAddress, rfc822Name

Own Original Figure (2019)

CA-CERTIFICATE SELECTED EXAMPLE

Data: Version: 3 (0x2)

Serial Number: 24:c8:7a:45:67:a6:1c:99 Signature Algorithm : sha256WithRSAEncryption

Issuer : C=AT, ST=Vienna , O=Course Test, CN=Course

Validity

Not Before: May 1 05:27:16 2021 GMT

Not After : May 2 05:27:16 2026 GMT Subject : C=AT, ST=Vienna , O=Course Test, CN=Course Subject Public Key Info :

Public Key Algorithm : rsaEncryption RSA Public Key: (4096 bit)

Modulus (4096 bit):

00:c7:a6:87:d1:e6:93:25:33:67:10:d8:a4:22:40:

Exponent : 65537 (0 x10001) X509v3 extensions :

X509v3 Subject Key Identifier : AA:76:4A:C3:87:BD:32:33:AC:A4:C3:75:A8:23:A5:97:B3:2B:11:CC

X509v3 Authority Key Identifier :

keyid :EA:89:8D:B4:94:AC:77:D8:ED:ED:DC:11:B8:13:A4:9A:B6:4B:61:B2

DirName :/C=AT/ST=Vienna/O=Test serial :75:A4:AA:C3:34:E3:A7:87

Course/CN=Test

X509v3 Basic Constraints:CA:TRUE

X509v3 Key Usage :

CertificateSign , CRLSign

Signature

46:78:33:a6:d4:43:cd:98:3d:23:f2:d6:84:3a:dd:28:65:a3 :.....

EXTENSIONS FOR END-USER CERTIFICATES

SELECTED X.509 EXAMPLES

X509v3 extensions :

X509v3 Subject Key Identifier:

43:6A:22:53:C3:A7:7A:AB:77:36:45:54:7A:98:BE:BB:3C:8F:F1:23

X509v3 Authority Key Identifier :

keyid :EC:33:7A:D6:23:CA:55:B1:EC:CE:56:78:AC:EF:33:54:C5:26:17:B5

X509v3 Basic Constraints:CA:FALSE

X509v3 Key Usage:

digitalSignature, nonRepudiation, dataEncipherment, keyEncipherment

X509v3 Extended Key Usage: critical TLS Web Server Authentication

X509v3 CRL Distribution Points:

URI: <http://the.crl.server.com>

PROOF OF POSSESSION OF THE PRIVATE KEY¹

CERTIFICATE SIGNING REQUEST (CSR)

Proof of possession via a self-signed PKCS#10^{2,3,4} Certificate Signing Requests (CSR) by the OESTERREICH.GV.AT-App that includes the public key of the OESTERREICH.GV.AT-App

Protected transmission of the CSR via TLS to a Binding Service to derive the binding of the user to the app.

¹ cf. pp. 24-25 Zertifizierungsrichtlinie / Certificate Policy, oesterreich.gv.at, Version 1.0, 11.02.2019

² PKCS#10 – Public Key Cryptography Standards

³ cf. IETF RFC 2986 PKCS #10: Certification Request Syntax Specification Version 1.7, November 2000

⁴ Annotation: A data type of messages that are transmitted to a certification authority (CA) to request the certification of a public key.

PEM vs. DER FORMAT

DIGITAL CERTIFICATE REPRESENTATION

PEM (Privacy Enhanced Mail) format is favored for its simplicity and compatibility, making it a popular choice for web servers, email systems, and other applications requiring certificate management. It can also store other types of cryptographic data, such as private keys and certificate chain files, with each type having its own specific begin and end delimiters

DER (Distinguished Encoding Rules) format is purely binary and thus more compact and less human-readable. DER is widely used for cryptographic certificates and keys, especially in environments where the textual overhead of PEM is undesirable or where binary data is preferred for its efficiency in storage and transmission.

PRIVACY ENHANCES MAIL (PEM)

AN OVERVIEW

Widely used format for storing and transmitting cryptographic keys, certificates, and other data

- Base64 Encoding: PEM format encodes binary data into text using Base64 encoding. This makes the certificate easily readable and writable in text editors and compatible with various transmission protocols that may not support binary data.
- Delimiters: A PEM-formatted certificate is enclosed between a “begin certificate” line and an “end certificate” line. (*cf., slide 49*)
- Content: Between these delimiters is the Base64-encoded representation of the certificate’s binary data. This includes the certificate’s public key, the issuing Certificate Authority (CA), validity dates, and other relevant information encoded in the X.509 standard format for public key certificates.

Agenda

Introduction

Basics about Symmetric Cryptosystems

Fundamental Principles about Asymmetric Cryptography and PKI

Cryptographic Hash Functions and Symmetric Authentication

Summary

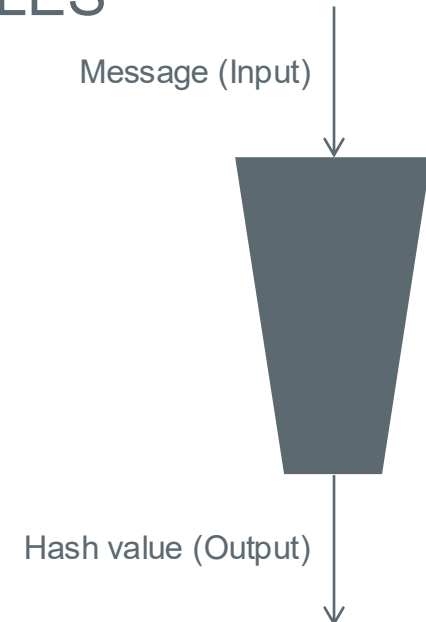
APPLICATIONS OF HASH-FUNCTIONS

PRACTICAL SCENARIOS | SELECTED EXAMPLES

- Data authentication
- Digital signatures
- Randomization
- Password protection
- Payments
- [...]

Examples of hash primitives provide Integrity

- MD5, SHA-1 consider to be broken
- SHA-2 family (*including: SHA-224, SHA-256, SHA-384 und SHA-512*)
- SHA-3 family (*including: SHA3-224, SHA3-256, SHA3-384, SHA3-512*)
- Whirlpool, Blake2, ... etc.



BUT WHAT IS A HASH-FUNCTION?

EXPLANATION

Hash functions (h) are one-way functions such that

- $y = h(x)$ (*computationally easy to derive*)
- $x = h^{-1}(y)$ (*computationally hard to derive i.e., computationally impossible*)
- $h(x) \neq h(x')$ (*collision resistance; hard to find (x, x') with $x \neq x'$ s.t. $h(x) = h(x')$*)

Not all hash functions are collision resistant !

A hash (*value*) is a (*digital*) fingerprint of a document

2 documents with different content should not have the same fingerprint

SECURITY PROPERTIES OF CRYPTOGRAPHIC HASH FUNCTIONS

Security Properties of Cryptographic Hash Functions

- **Collision resistance.** Means that it should be computationally infeasible to find two messages, such that they have the same hash value derived.
- **Second preimage resistance.** For a given message, it should be computationally infeasible to find a second message, such that they have the same hash value.
- **Preimage resistance.** For a given hash value derived, it should be computationally infeasible to find any message, such that it has the given hash value.

SECURITY CRITICAL BUILDING BLOCKS

HASH FUNCTIONS

- Digital Signatures
- Password storage
- Bit commitment schemes
- Key derivation functions
- Message authentication codes
- Tags of files (i.e., to detect changes)
- Cryptographic Tokens
- ...

EXAMPLES FOR TOKENS

PROPERTIES

Examples for Tokens

- Smart card
- Hardware Security Modules
- USB tokens
- Trusted Platform Module (TPM)

Support for on-token key lifecycle

- Key never leaves token
- Key confidentiality
- Key integrity
- Control of cryptographic token

WHY HASH-FUNCTIONS IN PASSWORD FILES?

VERIFICATION AND PREIMAGE DERIVATION

The pair (*username*, *passphrase*) are NOT stored plain in password files

Assume an attacker obtains access to the passwordfile

Case1: (*user_id*, *passphrase*) both are accessible in plaintext form

Case2: (*user_id*, $h(\textit{passphrase})$) the attacker has to derive a preimage

WHAT DOES COLLISION RESISTANCE IMPLY?

PRACTICAL EXAMPLE

- 1) Assume the source code x and x'
- 2) Submit the source code x to the operator of the app store
- 3) The operator of the app store inspects x
- 4) The operator approves x
- 5) The operator electronically signs x
- 6) If it holds: $h(x) = h(x')$
- 7) A malicious attacker distributes x' with the valid hash / signature

ATTACKS ON HASH FUNCTIONS

SELECTED EXAMPLES

- Guess a (second) preimage
- Birthday attack (collision)

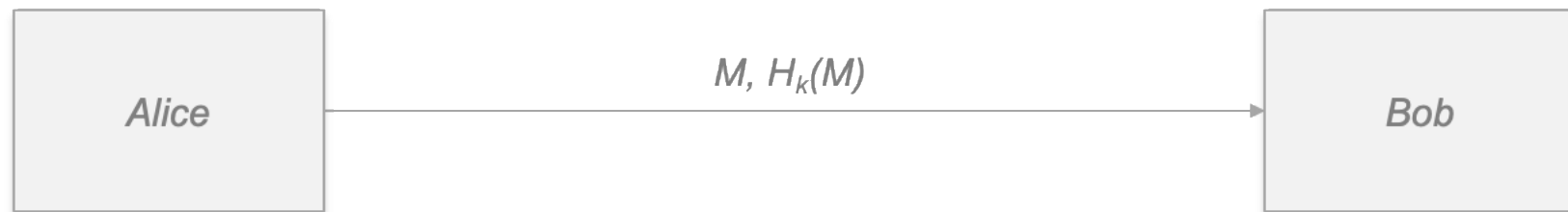
Mainly depends on the size (*i.e.*, n bits) of the hash value

SYMMETRIC AUTHENTICATION

MESSAGE AUTHENTICATION

Keyed-hash function

- HMAC (*applied in SSL/TLS, IPSec*)
- CMAC (*in conjunction with AES or TDES*)



¹ HMAC – Hash Message Authentication Code

² CMAC – Cipher-Based Message Authentication Code

Agenda

Introduction

Basics about Symmetric Cryptosystems

Fundamental Principles about Asymmetric Cryptography and PKI

Cryptographic Hash Functions and Symmetric Authentication

Summary

ENCRYPTION VS. SIGNATURE

WHAT IS THE DIFFERENCE?

Encryption

- Confidentiality
- Keypair (*sender and receiver*)
- K_e not necessarily the same as K_d

General Process

- Alice computes K_e
- Alice transmits $K_e(M)$
- Bob decrypts $K_d(K_e(M))$

Signature

- Authenticity
- Integrity
- Non-repudiation
- Keypair (K_V, K_S)

General Process

- Signatory signs $K_S(M)$
- Verifier obtains K_V
- Verifier checks $K_V(K_S(M))$

SOME HINTS

- Digital signatures are used in a variety of products
- GnuPG allows digitally signing and encrypting emails
- Java cryptographic architecture allows digital signatures via API
- Microsoft .Net allows digital signatures via API
- XML signature can be used
- Emails not very much secure (*it was not designed with security in mind*)

